

Home

Library

Profile

Stories

Stats

Following

Python in Plain English

Dev Genius

Level Up Coding

The Useful Tech

Top Python Libraries

Generative AI

Realworld AI Use Cases

AI Mind

Deep concept

The Mac Alchemist

More

Member-only story

AI

12 Claude Skills Every Developer Should Be Using Right Now

Backed by Real-World Use Cases



Vijayasekhar Deepak

Follow

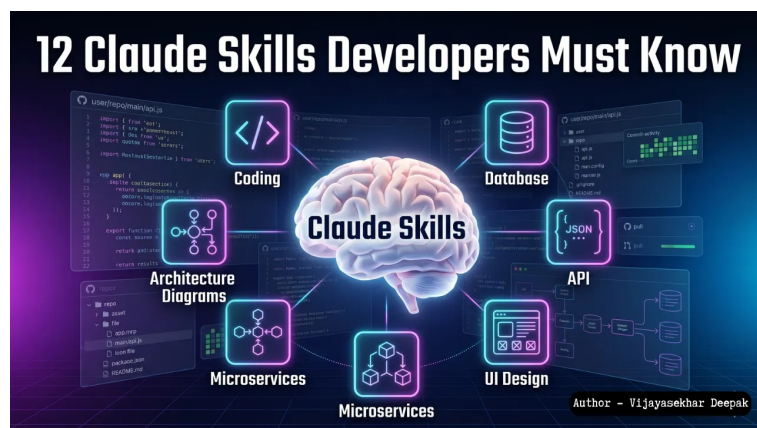
9 min read · 5 days ago

263

1



Not a Member? Read for FREE [here](#).



There's a version of you that opens Claude, types a

vague prompt, gets a mediocre result, and closes the tab frustrated.

Then there's the version of you that hands Claude a *skill*, a structured, reusable instruction set that turns a general-purpose AI into a specialist who knows your codebase, your stack, and your standards.

That second developer ships faster, writes better code, and spends their mental energy on problems that actually matter.

This article is about becoming that developer.

These are the 12 Claude skills that cover everything from designing airtight APIs to building entire frontend systems, orchestrating microservices, and even creating *new* skills from scratch and these skills I use regularly in my regular job and for my personal projects.

Let's get into it.

Note: I'm using Claude's name because they introduced it, but now every AI agent uses it.

What Is a Claude Skill, Exactly?

A Claude skill is a `SKILL.md` file, a structured

markdown document that tells Claude *how to behave* for a specific task. Think of it as a system prompt on steroids, packaged as a reusable module you can drop into any Claude-powered workflow.

Skills give Claude:

- A **role** (e.g., “You are a senior database optimizer”)
- A **methodology** (step-by-step thinking process)
- **Constraints** (what to avoid, what to enforce)
- **Output formats** (specific schemas, file structures, or patterns)

The result? Claude stops being a chatbot and starts being a teammate who knows exactly what they’re doing.

Now, the 12 skills every developer needs.

1. Obsidian Skills — Your Second Brain, Now AI-Powered

 [obsidian-skills by kepano](#)

If you’re a developer who lives in Obsidian for documentation, architecture notes, or technical journaling, this skill changes everything.

The Obsidian Skills project by Steph Hou (creator of the Minimal theme) creates a bridge between your personal knowledge graph and Claude.

What it does in practice:

Imagine asking Claude to suggest a database schema, but instead of pulling from generic best practices, it references your own note on “preferred patterns for multi-tenant SaaS” sitting in your vault. That’s the superpower here.

Best for: Solo developers, indie hackers, technical leads who document heavily.

2. Skill Creator — Build Skills That Build Skills

 [skill-creator by Anthropic](#)

This one is meta in the best possible way.

The Skill Creator skill teaches Claude how to *create other skills*. You describe a workflow, a role, or a repetitive task, and Claude outputs a production-ready `SKILL.md` that you can immediately deploy.

The workflow:

You → **describe** a task you do repeatedly

↓
Skill Creator → generates a structured SKILL.md
↓
You → drop it into your Claude workflow
↓
Claude → behaves like an expert every single time

Why it matters:

Skills compound. The more you build, the more leverage you have. Skill Creator is the foundation of a developer's personal AI toolkit.

Best for: Teams who want consistent AI behaviour, developers with recurring workflows.

3. NotebookLM-py — RAG for Your Technical Docs

 [notebooklm-py by teng-lin](#)

Google's NotebookLM is impressive. But it's locked behind a UI, limited in customization, and not plugged into your dev workflow.

NotebookLM-py changes that. It's a Python implementation of NotebookLM-style RAG (Retrieval-Augmented Generation) that you can wire directly into Claude, giving it the ability to reason over your internal documentation, codebases, RFCs, and runbooks.

The developer use case:

Spin this up on your team's Confluence export or internal wiki. Connect it to Claude. Now every developer on your team has a senior engineer-level understanding of the codebase on day one.

Best for: Teams with large internal documentation, onboarding workflows, and technical knowledge bases.

4. iOS Simulator Skill — Ship Mobile Features Without the Guesswork

 [ios-simulator-skill by conorluddy](#)

Mobile developers know the pain: you write Swift code, Claude suggests a fix, and then... you have to context-switch into Xcode to see if it actually works in the simulator.

The iOS Simulator Skill brings that feedback loop directly into your Claude workflow. It gives Claude structured awareness of simulator states, device contexts, and iOS-specific behaviours, so its suggestions are grounded in how iOS apps actually behave, not how they theoretically should.

Why this matters for teams:

When your whole iOS team uses this skill, code review feedback becomes more precise. Claude stops suggesting things that only work in theory and starts suggesting things that work on a real iPhone 15 Pro Max in dark mode at 3x resolution.

Best for: iOS developers, mobile engineers, Swift/SwiftUI teams.

5. Stitch Skills (Google Labs) — Design System Intelligence

 [stitch-skills by google-labs-code](https://stitch-skills-by-google-labs-code)

Google Labs' Stitch is their AI-powered design-to-code system. The Stitch Skills library extends Claude with deep knowledge of design tokens, component hierarchies, and the Stitch methodology for bridging Figma designs and production code.

The developer superpower:

You paste a Figma component description. Claude, equipped with Stitch Skills, outputs code that doesn't just look right but is architecturally aligned with your design system. Spacing scales, typography hierarchies, and color tokens all map correctly.

Best for: Frontend developers, design-system engineers, full-stack teams with Figma workflows.

6. Frontend Design Skill — Never Ship Ugly UIs Again

 [frontend-design by Anthropic](#)

This is one of Anthropic's own first-party skills, and it's exceptional.

The Frontend Design skill transforms Claude from a code-generator into a *designer-developer hybrid*. It doesn't just write working HTML/CSS/React, it writes code with genuine aesthetic intentionality.

What makes this different:

Most AI-generated UI looks the same: Inter font, purple gradient, white background, card-based layout. The Frontend Design skill explicitly rejects these defaults. It forces Claude to:

- Pick a **bold aesthetic direction** before writing a single line of code
- Choose **distinctive, context-appropriate typography** (not Inter or Roboto)
- Build **motion and micro-interactions** that feel intentional
- Create **spatial composition** with personality — asymmetry, overlap, negative space

Best for: Any developer who ships user-facing products and cares about design quality.

7. MCP Builder Skill — Build Model Context Protocol Servers Fast

 [mcp-builder by Anthropic](#)

MCP (Model Context Protocol) is Anthropic's open standard for connecting Claude to external tools, APIs, and data sources.

Building MCP servers from scratch is non-trivial. The MCP Builder skill makes it approachable.

What it does:

Guides Claude through architecting and generating a complete MCP server, with proper tool definitions, error handling, authentication patterns, and connection logic based on your use case description.

A real workflow:

```
Developer: "I need an MCP server that connects Claude to
           PostgreSQL database and lets it run read-only

MCP Builder: Generates server scaffold, tool definitions,
           connection pooling logic, query sanitization
           and a README - ready to deploy
```

Why this is a game-changer:

MCP is the future of Claude integrations. Teams that build their internal MCP servers now will have AI tooling that's deeply wired into their actual systems — not just generic web browsing. This skill is the on-ramp.

Best for: Platform engineers, DevOps teams, developers building Claude-powered internal tools.

8. Software Architecture Skill — Think Before You Code

 [software-architecture by davila7](#)

The best developers know that the most expensive code is the code written in the wrong direction. Architecture decisions made at 9 AM on a Monday can haunt a team for three years.

The Software Architecture skill gives Claude a structured methodology for *thinking before building*, evaluating tradeoffs, modelling data flows, identifying failure modes, and producing architecture documentation before a line of implementation code is written.

What Claude does with this skill:

- Evaluates architectural patterns (monolith vs microservices, event-driven vs request-response)
- Produces C4 model-style diagrams in text/Mermaid format
- Identifies scalability bottlenecks and single points of failure
- Recommends technology choices with explicit tradeoff reasoning

The senior engineer framing:

This skill doesn't just tell you *what* to build. It explains *why* one approach beats another for your specific context — team size, traffic patterns, consistency requirements, and budget constraints.

Best for: Tech leads, senior engineers, startup CTOs making foundational decisions.

9. API Designer Skill — APIs Your Consumers Will Actually Love

 [api-designer by Jeffallan](#)

Bad API design is a tax. Every inconsistency, every poorly named endpoint, every missing error code is a support ticket, a confused integration partner, or a midnight incident.

The API Designer skill turns Claude into a senior API architect who lives and breathes REST conventions, OpenAPI specs, versioning strategies, and developer experience.

What it produces:

- RESTful endpoint design with consistent naming conventions
- Complete OpenAPI/Swagger specs, ready to import into Postman or Stoplight
- Error response schemas that actually help consumers debug
- Versioning and deprecation strategies baked in from day one
- Authentication flow recommendations (OAuth 2.0, API keys, JWTs) based on use case

Best for: Backend developers, API-first teams, platform engineers, developers building SDKs.

10. Database Optimizer Skill — Stop Leaving Performance on the Table

 [database-optimizer by Jeffallan](#)

Slow queries are silent killers. They're fine at 100 users. They're a P0 incident at 100,000.

The Database Optimizer skill equips Claude with the mindset of a DBA who's seen everything: N+1 queries, missing indexes, overloaded joins, inefficient aggregations, and schema designs that made sense until they didn't.

What Claude analyzes with this skill:

- Query execution plans and index usage
- Schema normalization vs. denormalization tradeoffs
- Partitioning strategies for large tables
- Connection pooling recommendations
- Caching layer suggestions (Redis, Memcached) based on access patterns

Why this matters:

Most database performance issues are solved not by throwing more hardware at the problem but by understanding what the database is actually doing.

Best for: Backend developers, full-stack engineers, and anyone who writes SQL regularly.

11. Architecture Designer Skill — From Whiteboard to Production Blueprint

architecture-designer by Jeffallan

Distinct from the Software Architecture skill (which focuses on decision-making), the Architecture Designer skill focuses on *deliverables*, the actual blueprints, diagrams, and documentation that a team needs to build from.

What it generates:

- System context diagrams (who talks to what)
- Component architecture docs with interface contracts
- Data flow diagrams for complex multi-service interactions
- Infrastructure architecture (cloud resource layout, networking, security boundaries)
- ADRs (Architecture Decision Records) that capture *why* choices were made

Best for: Solution architects, technical leads, teams preparing for system design reviews.

12. Microservices Architect Skill — Tame the Distributed Systems Beast

microservices-architect by Jeffallan

The Microservices Architect skill gives Claude the mental models of a distributed systems veteran, someone who's dealt with network partitions, eventual consistency, saga patterns, and the organizational realities of Conway's Law.

What Claude reasons about with this skill:

- **Service boundary definition:** where to split, where not to split (the “cell vs. organ” metaphor)
- **Communication patterns:** synchronous REST/gRPC vs. async messaging (Kafka, RabbitMQ) and when each is appropriate
- **Saga patterns** for distributed transactions that span multiple services
- **Service mesh considerations** (Istio, Linkerd) for observability and traffic management
- **Deployment topology:** how services should be packaged, deployed, and scaled independently

Best for: Senior engineers, platform teams, and architects designing systems that need to scale with organizational growth.

Putting It All Together: A Developer's Skill Stack

You don't need all 12 on day one. Here's a

recommended progression:

Start here (Week 1):

- Skill Creator: learn how skills work by building one
- Frontend Design: immediate visual quality improvement
- API Designer: cleaner interfaces from your next feature

Go deeper (Month 1):

- Software Architecture: better decisions before you write code
- Database Optimizer: performance gains in your current system
- MCP Builder: start connecting Claude to your actual tools

Level up (Ongoing):

- Architecture Designer + Microservices Architect: for complex system design
- Obsidian Skills + NotebookLM-py: for building your AI-enhanced knowledge system
- Stitch Skills + iOS Simulator: for specialized

The Developer Who Uses Skills vs. The Developer Who Doesn't

Six months from now, there will be a visible gap between developers who learned to use Claude effectively and those who didn't.

It won't be about who uses AI. Everyone will use AI. The gap will be about *how*, whether developers treat Claude as a magic text box or as a configurable, specialized, deeply capable collaborator.

Skills are how you cross that gap.

Every skill in this list represents someone solving the "Claude gives generic answers" problem in a specific domain. They did the work of encoding expertise, methodology, and standards into a reusable format. All you have to do is pick them up and use them.

So pick them up and start it today.

Thank You for Reading!

I hope you found it helpful and informative. If you have any questions or feedback, feel free to leave a comment below. Your support and engagement mean a lot to me.

If you enjoyed this article and would like to support my work, consider buying me a coffee. Your contributions help me to keep creating valuable content.

If you enjoyed this, consider buying me a coffee! ☕

I appreciate your support. See you in the next blog!

Happy Coding!

AI

Claude Code

Claude Skills

Coding

Web Development



Written by Vijayasekhar Deepak

Follow



1.2K followers · 323 following

Hi 🙋, I write stuffs around Technology 🌐,
Development 🧑‍💻, Software Industry Trends 🖥️,
and Innovations #Quote: Change is Only
CONSTANT 🌀

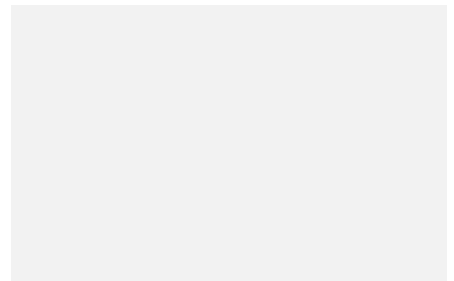
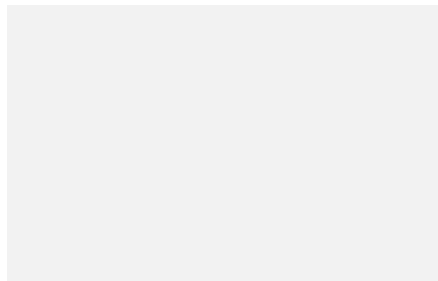


[Redacted text]

[Redacted text]

[Redacted text]

[Redacted text]



[Redacted text]

